

Singleton

Todo el mundo sabe que las fábricas de chocolate modernas cuentan con calderas de chocolate controladas por ordenador.

La función de la caldera es recibir el chocolate y la leche, llevarlos a ebullición y luego pasarlos a la siguiente fase de la elaboración de las tabletas de chocolate.

Aquí tenéis la clase de controlador para la caldera de chocolate industrial de Choc-O-Holic, Inc.

Echadle un vistazo al código; veréis que se han esforzado mucho por evitar problemas, como vaciar 500 galones de mezcla sin hervir, llenar la caldera cuando ya está llena o hervir una caldera vacía.

```
public class ChocolateBoiler
{
    private static ChocolateBoiler? _uniqueInstance;

    public static ChocolateBoiler Instance
    {
        get
        {
            if (_uniqueInstance == null)
            {
                Console.WriteLine("Creating unique instance of Chocolate Boiler");
                _uniqueInstance = new ChocolateBoiler();
            }
            Console.WriteLine("Returning instance of Chocolate Boiler");
            return _uniqueInstance;
        }
    }

    public bool Empty { get; private set; }
    public bool Boiled { get; private set; }

    private ChocolateBoiler()
    {
        Empty = true;
        Boiled = false;
    }

    public void Fill()
    {
        if (Empty)
        {
            Empty = false;
            Boiled = false;
            // fill the boiler with a milk/chocolate mixture
        }
    }
}
```

```
public void Drain()
{
    if (!Empty && !Boiled)
    {
        // drain the boiled milk and chocolate
        Empty = true;
    }
}

public void Boil()
{
    if (!Empty && !Boiled)
    {
        // bring the contents to a boil
        Boiled = true;
    }
}
}
```

Choc-O-Holic ha hecho un buen trabajo evitando que ocurran cosas malas, ¿no crees?

Claro que, probablemente sospechas que si dos instancias de ChocolateBoiler se descontrolan, pueden ocurrir cosas muy malas.

¿Cómo podrían salir mal las cosas si se crea más de una instancia de ChocolateBoiler en una aplicación?

Hacer una lista de 3 cosas que podrían salir mal.

- 1.
- 2.
- 3.

El patrón de diseño Singleton

El patrón Singleton garantiza que una clase tenga una sola instancia y proporciona un punto de acceso global a ella.

No hay grandes sorpresas. Pero analicemos esto con más detalle:

¿Qué sucede realmente aquí?

- Tomamos una clase y le permitimos administrar una única instancia de sí misma.
- También impedimos que cualquier otra clase cree una nueva instancia por su cuenta.
- Para obtener una instancia, hay que acceder a través de la propia clase.

Además, proporcionamos un punto de acceso global a la instancia: ***siempre que necesites una instancia, simplemente consulta la clase y te devolverá la única instancia.***

Como has visto, podemos implementar esto para que el Singleton se cree de forma diferida, lo cual es especialmente importante para objetos que consumen muchos recursos.

Problemas...

Parece que la caldera de chocolate nos ha fallado.

A pesar de que mejoramos el código usando el patrón Singleton clásico, el método `fill()` de la caldera de chocolate comenzó a llenarla incluso cuando ya estaba hirviendo leche y chocolate.

¡Eso son 500 galones de leche (y chocolate) derramados! ¿Qué pasó?

¡No sabemos qué pasó! El código Singleton funcionaba correctamente.

Lo único que se nos ocurre es que simplemente agregamos algunas optimizaciones al Controlador de la Caldera de Chocolate que utiliza múltiples hilos.

¿Podría la adición de hilos haber causado esto?

¿No es cierto que una vez que hemos establecido la variable

uniqueInstance a la única instancia de ChocolateBoiler, todas las llamadas a `getInstance()` deberían devolver la misma instancia? ¿Verdad?

Tenemos dos hilos, cada uno ejecutando este código.

Tu tarea consiste en explorar y determinar si existe algún caso en el que dos hilos puedan acceder a objetos de caldera diferentes.

Sugerencia: solo necesitas observar la secuencia de operaciones en el método `get()` de la propiedad Instance y el valor de `_uniqueInstance` para ver si se superponen.

Para evitar que mas de un hilo pueda acceder a un recurso aún cuando ese recurso esté siendo usada hay que aplicar sincronización.

La sincronización de subprocesos en C# es un mecanismo utilizado para coordinar la ejecución de múltiples subprocesos y garantizar que no accedan simultáneamente a recursos compartidos, lo que previene condiciones de carrera y corrupción de datos.

Pero, la sincronización es **cara**. Es decir, consume tiempo y recursos.

Punto importante, y en realidad es un poco peor de lo que imaginamos: la sincronización solo es relevante la primera vez que se ejecuta este método.

En otras palabras, una vez que hemos asignado a la variable `uniqueInstance` una instancia de Singleton, ya no necesitamos sincronizar este método.

Después de la primera ejecución, la sincronización es una sobrecarga totalmente innecesaria.